PROJECT RESILIENCE-ALPHA

Plan d'action du groupe : qui fait quoi, quand, et comment

5 personnes • Code gelé jeudi 22/10 à 17 h • Rendus jeudi avant 23 h 59 • Jury vendredi 23/10

1 Ce qu'on doit montrer

Une plateforme de paiement **codée par nous** se fait attaquer depuis un autre PC, **détecte** l'attaque avec du machine learning et **se répare toute seule**. On le prouve en faisant passer une nouvelle transaction après l'incident.



Tout est visible en direct sur le dashboard : avant, pendant et après l'incident.

Règle d'or : si quelqu'un intervient à la main pour réparer pendant la démo, on a **0/5** sur la remédiation. Et un conteneur qui redémarre ne suffit pas : il faut une **transaction qui réussit**.

2 Qui fait quoi

Pilier	Mission en une phrase	Nom
P1 - DEV	Coder la plateforme de paiement (front, connexion, API, base) et le dashboard de supervision	
P2 - INFRA	Tout déployer avec Docker Compose + Ansible, monter la supervision, coder le moteur de réparation automatique	
P3 - IA	Entraîner le modèle ML qui repère l'attaque, le brancher sur la réparation, faire l'assistant IA local	
P4 - CYBER	Préparer et lancer l'attaque depuis le 2e PC, installer les protections (WAF, secrets, durcissement, SOC)	
P5 - GOUV.	Analyse de risques, PCA/PRA, NIS2/DORA. Chef de projet : planning, dossier, pitch, clip	

3 La stack technique retenue

On prend **Python / FastAPI pour tous les services** : un seul langage, tout le monde peut lire le code des autres.

Composant	Techno	À quoi il sert
Front web	React	Page de connexion + page « faire un paiement »
auth-service	FastAPI + JWT	Connexion des utilisateurs fictifs, délivre un jeton
payment-api	FastAPI	Créer / lister des transactions. Expose /health et /metrics
Base de données	PostgreSQL	Utilisateurs, comptes et transactions **fictifs**
WAF	Nginx + ModSecurity (règles OWASP CRS)	**Seul point d'entrée exposé** sur le réseau. Filtre et peut bannir une IP
Secrets	HashiCorp Vault (ou SOPS si plus simple)	Mot de passe de la base, clé JWT, token du moteur de réparation
Supervision	Prometheus + Loki (Promtail) + Grafana	Métriques, journaux centralisés, vue SOC
ml-detector	Python + scikit-learn (Isolation Forest)	Lit les métriques toutes les 5-10 s et décide si c'est anormal
remediation-engine	FastAPI + docker-socket-proxy + Ansible	Reçoit l'alerte, applique la réparation, vérifie avec une transaction test
Dashboard	React + Tailwind	État des services, alertes, étapes de la réparation, dernière transaction test
Assistant IA	Ollama + LangChain + Chroma	Répond aux questions de l'opérateur à partir de nos documents (modèle de 8B max)

4 Les tâches précises de chacun

Chaque case cochée doit pouvoir être montrée. Si une tâche bloque plus d'une heure, on en parle au point d'équipe.

P1 - DEV : la plateforme de paiement et le dashboard

Responsable : P1

- □ **Base PostgreSQL** : tables *users*, *accounts*, *transactions* + un script qui crée 3-4 utilisateurs fictifs avec un solde.
- □ **auth-service** : *POST /login* (identifiant + mot de passe → jeton JWT), mots de passe hachés (bcrypt).
- □ **payment-api** : *POST /transactions* (vérifie le jeton et le solde, enregistre le paiement), *GET /transactions*, *GET /health*, *GET /metrics* (pour Prometheus).
- □ **Front React** : page de connexion → page de paiement (montant, destinataire) → message « paiement accepté ».
- □ **Dashboard React + Tailwind** : un voyant par service (vert / orange / rouge), la liste des alertes, les étapes de la réparation en direct, le résultat de la dernière transaction test.
- □ Écrire la **documentation de l'API** (FastAPI la génère avec /docs) et quelques tests automatiques.

Terminé quand : on peut se connecter et payer depuis le navigateur, et le dashboard affiche l'état réel des services.

P2 - INFRA : le déploiement, la supervision et la réparation automatique

Responsable : P2

- □ **docker-compose.yml** avec tous les services, deux réseaux (*public* pour le WAF, *interne* pour le reste). **Seul le WAF publie un port**.
- □ **Réseau** : point d'accès mobile Windows « RESILIENCE-G<n> », PC Hôte en 192.168.137.1.
- □ **Supervision** : Prometheus récupère les /metrics, Promtail envoie tous les journaux dans Loki, Grafana affiche un tableau de bord.
- □ **Playbooks Ansible** : *deploy.yml* (tout installer), *reset.yml* (remettre la démo à zéro), *remediate.yml* (les actions de réparation).
- □ **remediation-engine** : reçoit *POST /alert* (avec un token), choisit l'action selon le type d'attaque (bannir l'IP au WAF, relancer le service, restaurer la base), puis **lance une transaction test** et annonce « service rétabli » seulement si elle réussit.
- □ Le moteur passe par un **docker-socket-proxy** limité aux actions utiles, jamais par le socket Docker brut. Toutes ses actions sont journalisées.
- □ **README** : comment installer et relancer la plateforme sur un autre PC.

Terminé quand : en lançant l'attaque, la réparation se déclenche seule et la transaction test passe.

P3 - IA : la détection par machine learning et l'assistant

Responsable : P3

- □ **Générateur de trafic normal** (Locust ou un script Python) : de faux utilisateurs qui se connectent et paient en continu.
- □ **Jeu de données** : toutes les 10 s, relever dans Prometheus/Loki les requêtes/s, le taux d'erreurs, la latence, les échecs de connexion, le CPU. Enregistrer en CSV.
- □ **Entraîner un Isolation Forest** sur le trafic normal. Tester sur un enregistrement d'attaque. Noter les résultats (détections, faux positifs).
- □ **Service ml-detector** : boucle toutes les 5-10 s → calcule un score d'anomalie → l'expose à Prometheus → si anormal, envoie l'alerte au moteur de réparation.
- □ **Assistant RAG** : Ollama (Llama 3.1 8B ou Qwen 2.5 7B, ou un modèle 3B si le PC manque de RAM) + LangChain + Chroma, indexé sur nos procédures, le PCA/PRA et l'architecture.
- □ L'assistant est **en lecture seule** : il conseille l'opérateur, il ne répare rien et n'a aucun accès au système.

Terminé quand : l'attaque fait monter le score d'anomalie, l'alerte part toute seule, et le trafic normal ne déclenche pas de fausse alerte.

P4 - CYBER : l'attaque et les protections

Responsable : P4

- □ **Attaque principale** depuis le PC Attaquant : saturation de l'API de paiement (flood HTTP) **et/ou** force brute sur /login. Écrire un script lancé en une commande.
- □ Bonus si le temps le permet : une injection SQL que le WAF bloque.
- □ **WAF** : Nginx + ModSecurity avec les règles OWASP CRS, limitation du nombre de requêtes, liste d'IP bannies que le moteur de réparation peut modifier.
- □ **Secrets** : mot de passe de la base, clé JWT et token du moteur rangés dans Vault (ou chiffrés avec SOPS). Aucun secret dans le code.
- □ **Durcissement** : conteneurs sans droits root, base / Vault / Grafana non exposés, droits minimum pour chaque service.
- □ **Vue SOC** dans Grafana : échecs de connexion, requêtes bloquées par le WAF, alertes ML, actions du moteur de réparation.
- □ Vérifier que **personne d'autre** que le ml-detector ne peut appeler le moteur de réparation.

Terminé quand : l'attaque a un effet visible dans Grafana, le WAF journalise, et aucun service interne n'est accessible depuis le PC Attaquant.

P5 - GOUVERNANCE : risques, continuité, conformité et pilotage

Responsable : P5

- □ **EBIOS RM** (version courte) : ce qu'on protège (le paiement), qui pourrait attaquer et pourquoi, les scénarios d'attaque (dont **notre** attaque), la matrice gravité × vraisemblance, les mesures retenues.
- □ **PCA/PRA** : services critiques, objectifs de reprise (**RTO =** temps max d'arrêt, **RPO =** perte de données max), qui fait quoi, comment on restaure, et ce qui reste à risque. Comparer avec le **temps de reprise mesuré** pendant les tests.
- □ **NIS2 / DORA** : un tableau « exigence → ce que notre démonstrateur prouve → ce qu'un prototype de 4 jours ne peut pas prouver ».
- □ **Recommandations à la direction** : 3 à 5 décisions argumentées (budget, priorités, ce qu'on ferait en vrai en entreprise).
- □ **Chef de projet** : point d'équipe de 10 min matin et soir, suivi de ce document, rédaction du dossier, du PPTX et du script du clip.

Terminé quand : les documents parlent de NOTRE architecture et de NOTRE attaque, pas de généralités.

5 Le planning jour par jour

Jour	P1 - DEV	P2 - INFRA	P3 - IA	P4 - CYBER	P5 - GOUV.	Objectif du soir
LUNDI architecture	Choix des API (routes, données échangées). Schéma de la base.	Installer Docker sur le PC Hôte, monter le hotspot, créer le dépôt Git, squelette du docker-compose.	Choisir les indicateurs à surveiller. Installer Ollama (PC IA si besoin).	Choisir l'attaque. Préparer le PC Attaquant et ses outils.	Ateliers EBIOS avec toute l'équipe. Schéma d'architecture. **Faire valider par le coach.**	Architecture + EBIOS **validées par le coach**. Rôles fixés.
MARDI production	Base + auth-service + payment-api qui marchent. Début du front.	Tous les services dans Compose. Prometheus, Loki, Grafana branchés. Playbook *deploy.yml*.	Générateur de trafic. Collecte des données normales. Premier entraînement du modèle.	Script d'attaque. WAF en place. Secrets dans Vault ou SOPS.	Tableau NIS2/DORA. Objectifs RTO/RPO. Plan du dossier.	**Une transaction passe**. L'attaque se voit dans Grafana.
MERCREDI intégration	Front terminé. Dashboard (voyants, alertes, étapes de réparation).	Moteur de réparation + transaction test. Playbooks *reset* et *remediate*.	ml-detector en service, **branché sur le moteur de réparation**. Assistant RAG indexé.	Vue SOC. Durcissement. Contrôle des accès au moteur.	PCA/PRA rédigé. Recommandations. Rédaction du dossier.	**Premier cycle complet** : attaque → détection → réparation → transaction OK.
JEUDI recette et rendus	Corriger les bugs. Tests. Doc de l'API.	Tests de pannes (couper des services). README. Archive ZIP.	Ajuster le modèle (faux positifs). Résultats pour le dossier.	Rejouer l'attaque. Vérifier qu'aucun secret n'est dans l'archive.	Finaliser dossier + PPTX. **Tourner le clip** avec l'équipe.	**17 h : code gelé**. Démo répétée 10 fois. **Rendus avant 23 h 59**.
VENDREDI jury	Répond aux questions sur l'appli.	Lance la plateforme, garde le PC Hôte.	Répond aux questions ML / RAG.	Lance l'attaque depuis le PC Attaquant.	Mène le pitch, gère le chrono.	5 min de présentation + démo, puis 5 min de questions.

Point critique : la jonction ML → moteur de réparation (mercredi) bloque tout le reste. P2 et P3 s'accordent **dès mardi** sur le format exact de l'alerte (type d'attaque, score, IP source, heure).

6 Le déroulé de la démo du vendredi (5 min)

Temps	Qui	Ce qu'on montre
0:00 - 1:00	P5	Le contexte, ce qu'on a choisi de protéger, l'attaque retenue (en lien avec l'EBIOS)
1:00 - 1:30	P1	La plateforme en état normal : on fait un paiement, tout est vert sur le dashboard
1:30 - 3:30	P4 puis P2/P3	P4 lance l'attaque → le service se dégrade → le ML détecte → le moteur répare seul → **nouvelle transaction réussie**. Personne ne touche au PC Hôte.
3:30 - 5:00	P5 + P3	Temps de reprise mesuré vs RTO, ce que couvre NIS2/DORA, limites de la solution, recommandations
5 min	Tous	Questions du jury : chacun répond sur sa partie, mais tout le monde doit comprendre l'ensemble

7 Les rendus (dossier Workshop2026-M2-G<n>)

Fichier	Qui	Ce qu'il doit contenir
…-Dossier.pdf 8 à 12 pages	P5 + tous	1. Contexte et périmètre • 2. Architecture et flux • 3. L'attaque • 4. La détection ML (données, modèle, résultats, limites) • 5. La réparation automatique • 6. La sécurité • 7. EBIOS • 8. PCA/PRA • 9. NIS2/DORA • 10. Limites et recommandations • **11. Qui a fait quoi**
…-Pres.pptx	P5	Support court, aux couleurs de la charte (Montserrat, #261E48 / #E4E6F3 / #A1A6D4). Il accompagne la démo, il ne la remplace pas.
…-VidChrono.mp4	P5 + P2	60 s **sans coupe**, filmé avec OBS : terminal de l'attaquant + dashboard + Grafana à l'écran. Attaque → anomalie → réparation → transaction OK. Sous-titres autorisés.
…-Code.zip	P2	Tout le dépôt Git : code, docker-compose, Ansible, scripts d'attaque et de ML, tests, doc API, README. **Aucun secret** (fournir un .env.example).

8 Les pièges à éviter

- **Vouloir tout faire** : une seule attaque bien maîtrisée vaut mieux que trois qui plantent.
- **Brancher le ML trop tard** : c'est la pièce qui relie tout, elle doit tourner mercredi midi.
- **Ne pas répéter la démo** : avec le script *reset*, on la rejoue au moins 10 fois jeudi.
- **Trop charger le PC Hôte** (16 Go) : pas d'Elastic ni de Wazuh. Mettre Ollama sur un 3e PC si besoin.
- **Exposer des services internes** : base, Vault, Grafana admin et Prometheus restent sur le réseau interne.
- **Ne pas comprendre le code généré par IA** : le jury pose des questions dessus.
- **Oublier les règles** : émargement Edusign dans les 15 premières minutes de chaque demi-journée, aucune attaque hors de nos machines, aucune vraie donnée bancaire, aucun secret envoyé à une IA.